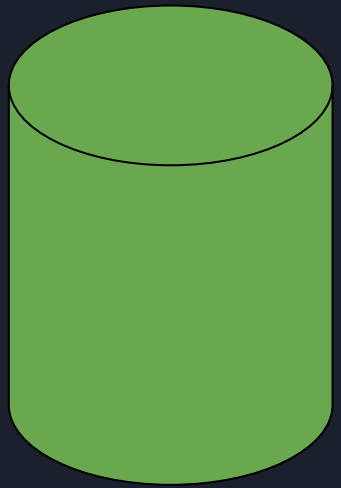
GESTIÓN DE DATOS

ADMINISTRACIÓN: PARTICIONES

RECORDERIS...



¿QUÉ APRENDEREMOS HOY?



- Qué es particionamiento en bases de datos
- Criterios para particionar
- Cuando particionar, cuando no
- Particionar acorde a motores de almacenamiento

PARA REFLEXIONAR...

¿Usted ha particionado algún medio de almacenamiento, como un disco duro o una memoria usb?



¿Por qué y para qué? ¿Bajo qué criterios?

(discusión verbal)

¿Qué significa "particionar"



DEFINICIÓN: Particionar

Dividir una tabla en segmentos más pequeños (¡tal como en un disco duro!)

A nivel lógico sigue siendo una sola tabla, pero a nivel físico son varios archivos separados.

PD: Si ya entendiste particionamiento en discos, ya entendiste particionamiento en bases de datos

Particionamiento: ¿Para qué?

- Reducir las operaciones de lectura y escritura en tablas que son muy grandes (y por ende, mejora la velocidad)
- Se reduce la probabilidad de pérdida de información en caso de un daño en los archivos
- Facilita la eliminación de datos antiguos o archivados
- *Pero…. mal usado no mejora… empeora!!! (así como con los índices)*

Particionamiento: forma gráfica

En un disco duro:



1 disco
4 particiones

En una base de datos:

id	first_name	last_name
1	SELENA	GOMEZ
2	KAROL	G
3	ANDRES	CEPEDA
4	MICHAEL	JACKSON

id	first_name	last_name
1	SELENA	GOMEZ
2	KAROL	G

id	first_name	last_name
3	ANDRES	CEPEDA
4	MICHAEL	JACKSON

1 tabla
2 particiones

Pregunta: ¿En cuál opción se busca más rápido?

Particionamiento: tipos

NOTA:

Lo importante no es aprenderse de memoria los tipos de particionamiento siguientes.

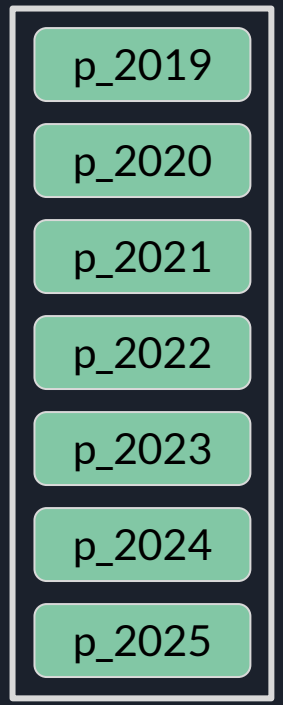
Lo importante es saber *cuando usar cual*

Particionamiento: tipos

POR RANGO (RANGE)

Se define una expresión que retorna un valor. Dependiendo del valor, el registro se ubica en una u otra partición.

```
create table singers(
  id int not null,
  first_name varchar(50),
  last_name varchar(50),
  birth_date date,
  gender enum('F','M','LGBTIQ+'),
  primary key(id)
)
partition by range(year(birth_date)) (
  partition p_2019 values less than (2019),
  partition p_2020 values less than (2020),
  partition p_2021 values less than (2021),
  partition p_2022 values less than (2022),
  partition p_2023 values less than (2023),
  partition p_2024 values less than (2024),
  partition p_2025 values less than (2025)
);
```

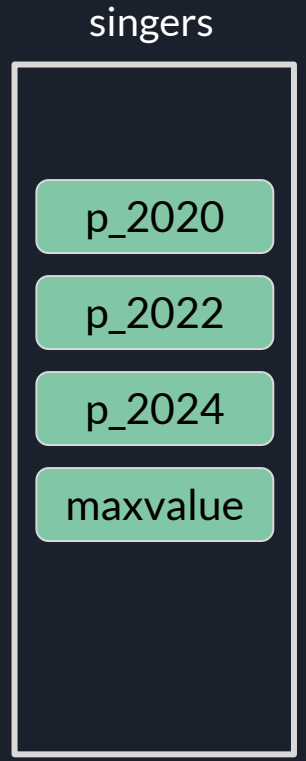


Particionamiento: tipos

POR RANGO (RANGE)

La expresión puede tener un valor por defecto llamado "maxvalue".

```
create table singers(
  id int not null,
  first_name varchar(50),
  last_name varchar(50),
  birth_date date,
  gender enum('F','M','LGBTIQ+'),
  primary key(id)
)
partition by range(year(birth_date)) (
  partition p_2020 values less than (2020),
  partition p_2022 values less than (2022),
  partition p_2024 values less than (2024),
  partition p_9999 values less than maxvalue
);
```



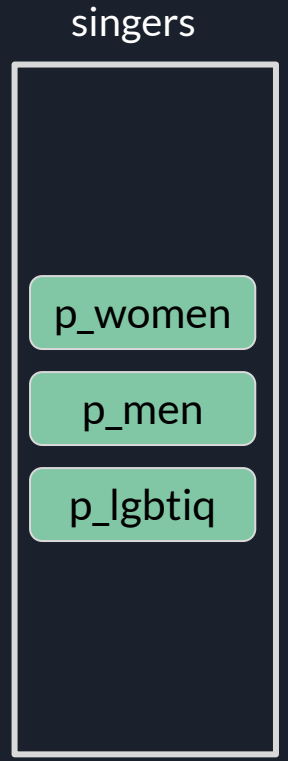
Particionamiento: tipos

POR LISTA (LIST)

Se define un valor fijo para un campo específico.

NOTA: El campo no necesariamente debe ser ENUM.

```
create table singers(
  id int not null,
  first_name varchar(50),
  last_name varchar(50),
  birth_date date,
  gender enum('F','M','LGBTIQ+'),
  primary key(id)
)
partition by list(gender) (
  partition p_woman values in ('F'),
  partition p_man values in ('M'),
  partition p_lgbtiq values in ('LGBTIQ+')
);
```



Particionamiento: tipos

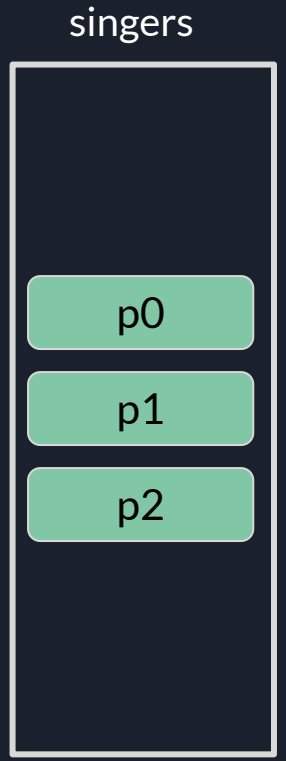
POR HASH (HASH)

Se define una cantidad de particiones y una expresión que debe retornar un entero.

El hash es el residuo de dividir el resultado de la expresión entre el número de particiones

```
create table singers(
  id int not null,
  first_name varchar(50),
  last_name varchar(50),
  birth_date date,
  gender enum('F','M','LGBTIQ+'),
  primary key(id)
)
partition by hash(month(birth_date))
  partitions 3
);

Ej: si birth_date es '1992-07-22':
  → expression = month('1992-07-22') = 7
  → expression % partitions = 7 % 3 = 1
  → El registro quedará en la partición p1
```



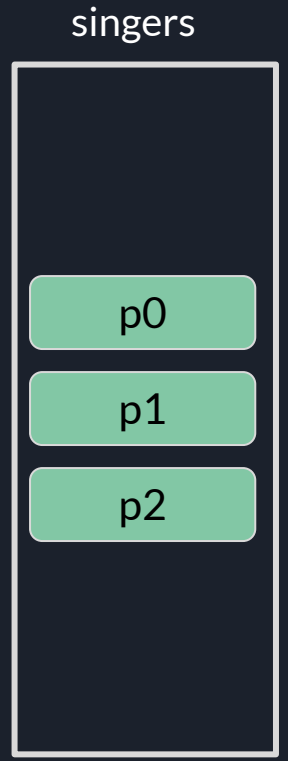
Particionamiento: tipos

POR HASH LINEAL (LINEAR HASH)

Similar al hash, pero el cálculo se hace usando funciones de potencia y logarítmicas.

Es mejor si hay que redistribuir las particiones a futuro

```
create table singers(
  id int not null,
  first_name varchar(50),
  last_name varchar(50),
  birth_date date,
  gender enum('F','M','LGBTIQ+'),
  primary key(id)
)
partition by linear hash(month(birth_date))
  partitions 3
);
```



Particionamiento: tipos

POR VALOR DE CAMPO (KEY)

Similar al hash, pero el cálculo lo hace internamente el DBMS basado en los valores de un campo o lista de campos.

```
create table singers(
  id int not null,
  first_name varchar(50),
  last_name varchar(50),
  birth_date date,
  gender enum('F','M','LGBTIQ+'),
  primary key(id)
)
partition by linear key(first_name)
  partitions 3
);
```

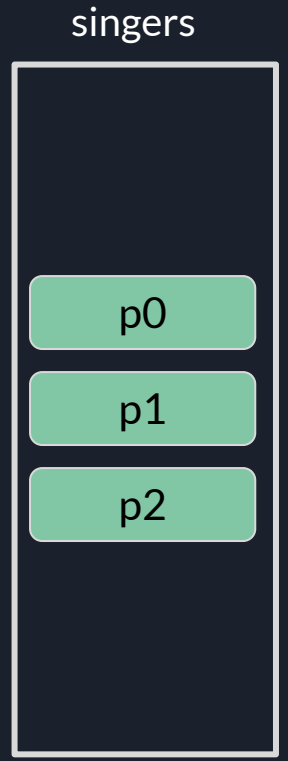


Tabla comparativa: particionamiento

Tipo	Observaciones
RANGE	Tiempos o fechas (históricos, auditorías, logs)
LIST	Lista ***corta*** de categorías o clasificaciones conocidas
HASH KEY	Distribución uniforme de los datos El dbms decide como particionar Útil si no hay un criterio claro o si solo se desea distribuir cargas

***Consejo**: procurar que el tipo de partición coincida con las búsquedas más frecuentes desde el front o desde el back (minimiza lecturas en disco)*

Particionamiento: discusión grupal



Que clase de particionamiento usar (y cual NO usar) para:

- Tabla de clientes con id, tipo_id, nombre, apellidos, género, dirección, ciudad, país
- Tabla de históricos de inventario generada por un trigger
- Tabla de inventario
- Registro de ingreso a la universidad usando los torniquetes
- Tienda de ropa en línea

Particionamiento: comandos

Caso	Comando
La tabla no existe y se va a crear particionada	*En las diapositivas anteriores se proporcionaron ejemplos para este caso*

La tabla ya existe, no está particionada y se debe particionar	`alter table singers`
	`partition by range(year(birth_date)) (`
	`  partition p_2020 values less than (2020),`
	`  partition p_2021 values less than (2022)`
	`);`

La tabla está particionada y se necesitan agregar más particiones	`alter table singers add partition(`
	`  partition p_2027 values less than (2027)`
	`);`

Particionamiento: comandos

Caso	Comando

reducir el número de particiones para tipo "hash" o "key":	`alter table singers coalesce partition 2;`
	
	`→ Si la tabla tenía 6 particiones, después de este comando quedará con 4`

Eliminar particionamiento (unificar la tabla de nuevo)	`alter table singers remove partitioning;`

Modificar rangos de una partición específica	`alter table singers reorganize partition into(`
	`  partition p_2020 values less than (2005)`
	`);`

Particionamiento: comandos

Caso	Comando
Eliminar una partición (sin eliminar la tabla)	`alter table singers drop partition p_2020;`

Redefinir particionamiento ("particionar la partición")	`alter table singers`
	`reorganize partition p_01 into(`
	`  partition p_01_a values less than (2010),`
	`  partition p_01_b values less than (2012)`
	`);`

Redefinir particionamiento ("unificar particiones")	`alter table singers`
	`reorganize partition p_01_a, p_01_b into(`
	`  partition p_01`
	`);`

IMPORTANTE!!!

El número máximo de particiones que pueden crearse en una tabla en MariaDB es 1024

Dato importante: más particiones ***no significa*** mejor rendimiento (de nuevo, ¿recuerdas el ejemplo del disco duro?)



IMPORTANTE!!!

Modificar particiones en un sistema funcionando y en una tabla grande es MUY costoso computacionalmente.

¿Porqué? Fácil: bloqueos, concurrencia (tema conocido, ¿verdad?)



SUPER IMPORTANTE!!!

Fórmula para un EXCELENTE rendimiento:

- **Índices óptimos**
- **Motores de almacenamiento acordes a su función**
- **Particiones adecuadas**
- **Buen diseño backend + dbms (concurrencia)**
- **Principio del mínimo privilegio**
- **Límites de ejecución por usuario**

EJERCICIOS

NOTA:

Para los siguientes ejercicios, se asume que se finalizó correctamente el ejercicio de la clase de motores de almacenamiento en el cual se solicitó insertar 5'000.000 de registros en cada tabla.

Si no ha terminado ese ejercicio, entonces los ejercicios de esta clase no tendrán ningún efecto práctico.

EJERCICIOS

Ejercicio 1:

Particionar las tablas acorde al siguiente criterio:

- t_aria: por rango de fecha "my_date", valores <2010, <2020, <2030 y maxvalue
- t_myisam: por rango de "my_value", valores <5000 y <10000
- t_innodb: por lista de "my_value_type"
- t_memory: por lista de "my_value_type"
- t_csv: por lista de "my_value_type"
- t_blackhole: por lista de "my_value_type"

Discusión grupal: ¿Sale algún error con cierto tipo de motores?

EJERCICIOS

Ejercicio 2

Acorde a lo explicado en la clase anterior, revisar en el sistema de archivos si después del particionamiento fueron generados nuevos archivos.

En caso de que sí, identificarlos y sacar conclusiones del porqué de la cantidad y del porqué de los tipos de archivos que cambiaron.

EJERCICIOS

Ejercicio 3

Repetir el ejercicio de la clase anterior, en el cual se consulta la cantidad de registros de números pares e impares. Comparar los tiempos de respuesta con la clase anterior, cuando las tablas no estaban particionadas y sacar conclusiones.

```
select  count(*)
from    t_xxxxx
where   my_value_type = 'ODD'

// repetir la sentencia sql cambiando ODD por EVEN.
```

EJERCICIOS

Ejercicio 4

Repetir el ejercicio de la clase anterior, para cantidad de registros con id entre 140.000 y 160.000 sobre cada una de las tablas. Tomar nota de los tiempos de respuesta en cada motor y compararlos con los resultados de cuando las tablas no estaban particionadas.

```
select  count(*)
from    t_xxxxx
where   id between 140000 and 160000;
```

EJERCICIOS

Ejercicio 5

Contar la cantidad total de registros en cada una de las tablas.

```
select  count(*)
from    t_xxxxx
```

Posteriormente, eliminar únicamente una partición en cada una de las tablas. Contar nuevamente la cantidad de registros y sacar conclusiones.

EJERCICIOS

Ejercicio 6

Eliminar el particionamiento en todas las tablas. Revisar nuevamente el sistema de archivos y concluir el porqué de los cambios en los archivos.

EJERCICIOS

Ejercicio 7

Sacar conclusiones respecto a los tiempos y a los archivos:

- ¿El particionamiento mejoró o empeoró los tiempos de respuesta?
- ¿Hubo algún motor de almacenamiento con mejor rendimiento al particionar?
- ¿Cuál fue la tabla que más rápido creó las particiones?
- ¿Hubo algún motor que no soportara particionamiento? ¿Porqué?
- Si se notaron archivos nuevos, ¿hubo alguna diferencia en el tamaño original VS el tamaño después de particionar?

¿PREGUNTAS?

